Documentation technique – Partie de Samuel

Gestion des convocations aux épreuves E5/E6 (CCF) – Laravel 12

Combari Samuel – BTS SIO SLAM – Lycée Saint-Joseph
Novembre 2026

0. Comment lire ce document

Ce document explique **ma partie** du projet : comment elle fonctionne, **dans quels fichiers** se trouve chaque morceau, et comment la tester ou la réparer. La partie d'Evan (épreuves, affectation des jurys, convocations, pages du candidat et des jurys, gestion des comptes en Livewire, modification d'un utilisateur, accueil, déconnexion, mise en forme) est expliquée dans son propre dossier.

Pour chaque fonctionnalité, tu trouveras toujours les mêmes rubriques :

- **À quoi ça sert** : une phrase simple ;
- **Fichiers concernés** : où regarder dans le projet ;
- **Que se passe-t-il, étape par étape** : le trajet d'une demande, du navigateur à la base de données ;
- **Tables utilisées** et **droit nécessaire** ;
- **Comment le tester**.

Rappel du schéma MVC de Laravel (utile pour tout le document) : le navigateur demande une adresse (une **route**, fichier `routes/web.php`) ; la route appelle un **contrôleur** (dossier `app/Http/Controllers`) ; le contrôleur utilise des **modèles** (dossier `app/Models`, un par table) pour lire ou écrire dans MariaDB, puis renvoie une **vue** Blade (dossier `resources/views`) qui fabrique la page HTML.

1. Ma partie en un coup d'œil

Les 15 fonctionnalités qui me sont attribuées (numérotation de la répartition) :

N°	Fonctionnalité	Où la trouver dans ce document
2	Inscription	4.1
3	Mot de passe oublié	4.3

N°	Fonctionnalité	Où la trouver dans ce document
4	Connexion	4.2
5	Limitation des tentatives de connexion	4.2
7	Réinitialisation du mot de passe	4.3
8	Menu adapté au rôle	4.5
9	Page Profil (et changement du mot de passe)	4.4
11	Liste des utilisateurs	4.6
12	Création d'un utilisateur	4.7
14	Paramétrage (rôles, statuts, genres)	4.8
23	Autorisations pilotées par la base	5
24	Les 5 rôles	5.1
25	Les statuts de compte	5.2
27	Documentation	7
29	Déploiement	8

Association non fonctionnelle de ma partie dans le MCD : **mcd_autoriser** (rôles ↔ actions).
Tables de ma partie : mcd_users, mcd_roles, mcd_statuts, mcd_genres, mcd_actions, mcd_autoriser (et mcd_utilisateurs, commune aux deux parties).

2. Les fichiers de ma partie

2.1 Dans le projet Laravel

Chemins relatifs à la racine du projet `pj_gestion_convocation_ccf_laravel/`.

Fichier	Rôle (en une phrase)
`routes/web.php`	Liste toutes les adresses du site et, pour chacune, le droit exigé (`auth` puis `can:`).
`bootstrap/providers.php`	Déclare les « fournisseurs » chargés au démarrage, dont `FortifyServiceProvider` (ajouté à la main).
`config/fortify.php`	Réglages de Fortify : champ de connexion (`email`), page d'arrivée (`/menu`), fonctions activées (inscription,

Fichier	Rôle (en une phrase)
	mot de passe oublié, changement de mot de passe), limitation des essais.
`app/Providers/FortifyServiceProvider.php`	Branche Fortify sur notre projet : vues à afficher, vérification de la connexion avec contrôle du statut, actions de création et de mot de passe.
`app/Actions/Fortify/CreateNewUser.php`	Valide le formulaire d'inscription et crée le compte et la fiche dans une transaction.
`app/Actions/Fortify/UpdateUserPassword.php`	Change le mot de passe depuis la page Profil.
`app/Actions/Fortify/ResetUserPassword.php`	Change le mot de passe après le lien reçu par e-mail.
`app/Actions/Fortify/PasswordValidationRules.php`	Règles de mot de passe communes (obligatoire, texte, règle par défaut de Laravel, confirmation).
`app/Http/Responses/RegisterResponse.php`	Après une inscription : déconnecte aussitôt le nouveau compte et affiche un message.
`app/Providers/AppServiceProvider.php`	Crée une **Gate** par action de la table mcd_actions (cœur des autorisations).
`app/Models/User.php`	Table mcd_users (le compte) ; méthode `hasPermission()`.
`app/Models/Utilisateur.php`	Table mcd_utilisateurs (la fiche) ; rôle, statut, genre ; méthode `hasPermission()`.
`app/Models/Role.php`	Table mcd_roles ; relation `permissions()` vers mcd_autoriser.
`app/Models/Permission.php`	Table mcd_actions ; relation `roles()` vers mcd_autoriser.
`app/Models/Statut.php`, `app/Models/Genre.php`	Tables mcd_statuts et mcd_genres.
`app/Http/Controllers/HomeController.php`	Page d'accueil `/` (renvoie vers `/menu` si déjà connecté).
`app/Http/Controllers/MenuController.php`	Page `/menu`.
`app/Http/Controllers/ProfilController.php`	Page `/profil`.
`app/Http/Controllers/UtilisateurController.php`	Liste (`index`), création (`create`, `store`) des utilisateurs.
`app/Http/Controllers/ParametrageController.php`	Page de paramétrage et ajout d'un rôle, d'un statut ou d'un genre.
`resources/views/auth/`	Vues de connexion, d'inscription, de mot de passe

Fichier	Rôle (en une phrase)
`*.blade.php`	oublié et de réinitialisation.
`resources/views/profil/show.blade.php`	Vue de la page Profil.
`resources/views/partials/menu.blade.php`	Barre de menu (liens affichés avec `@can` et `@canany`).
`resources/views/layouts/app.blade.php`, `auth.blade.php`	Cadres communs des pages (connectées / non connectées).
`resources/views/admin/utilisateurs/index.blade.php`, `create.blade.php`	Pages de la liste et de la création d'un utilisateur.
`resources/views/admin/parametrage.blade.php`	Page de paramétrage.
`lang/fr/auth.php`	Messages d'authentification en français (« Identifiants incorrects. », trop de tentatives…).
`README.md`, `docs/architecture.md`	Documentation du dépôt (installation, rôles, schémas, matrice des droits).

2.2 Hors du projet (déploiement)

Ces fichiers sont utilisés sur la machine virtuelle ; ils ne sont **pas** dans le code de l'application.

Fichier	Rôle
`setup_db_vm.sh`	Crée la base et son utilisateur, importe les 4 scripts SQL, fabrique le fichier `.env` (une seule fois, à la première installation).
`deploy.sh`	Déploie une nouvelle version : clone GitHub, dépendances, caches, permissions, bascule du lien `current`.
`ccf-convocations.conf`	Modèle du site Apache (PHP en mode FPM).
`creation.sql`	Création des tables, clés étrangères et triggers (dont mcd_actions et mcd_autoriser).
`init.sql`	Données de départ : 5 rôles, 3 statuts, 3 genres, compte administrateur, 13 actions, 15 autorisations.
`test.sql`	Jeu de données de test (comptes, jurys, épreuves, passages).
`ajout_password_reset.sql`	Crée la table mcd_password_reset_tokens (mot de passe oublié).
`.env`	Réglages propres à la machine (base de données, e-mail, clé de l'application). **Jamais dans Git** : il

Fichier	Rôle
	contient des secrets.

3. La base de données de ma partie

3.1 Les tables et leurs colonnes importantes

Table	Colonnes importantes	Remarque
mcd_users	id, name, email, password, remember_token	Le **compte de connexion**. Le mot de passe est **haché** (bcrypt), jamais en clair.
mcd_utilisateurs	id, nom, prenom, classe, numero_candidat, id_role, code_statut, code_genre	La **fiche métier**. Son `id` est **le même** que celui de mcd_users (c'est l'identifiant relatif du MCD, cardinalité 1,1(R)).
mcd_roles	id, code, nom, commentaire	5 rôles au départ : ADMIN, GEST, CAND, JURY_ENS, JURY_PRO.
mcd_statuts	code (1 lettre), nom, commentaire	A = actif, I = inactif, B = banni.
mcd_genres	code (1 lettre), nom, commentaire	Liste des genres proposés à l'inscription.
mcd_actions	id, nom, commentaire	Les 13 actions de l'application, nommées `ressource.action` (ex. `utilisateurs.create`). Modèle Laravel : `Permission`.
mcd_autoriser	id_role, id_action	Table de liaison : « ce rôle a le droit de faire cette action ». Clé primaire composée (id_role, id_action). 15 lignes au départ. **Pas de modèle** : on l'utilise à travers `belongsToMany`.

3.2 Comment les tables se relient

- Un **compte** (mcd_users) a **une fiche** (mcd_utilisateurs) de même identifiant : `User::utilisateur()` (relation `hasOne`).
- Une **fiche** a **un rôle**, **un statut** et **un genre** (relations `belongsTo`).
- Un **rôle** a **plusieurs actions**, et une **action** peut être donnée à **plusieurs rôles** : c'est la relation plusieurs-à-plusieurs, portée par **mcd_autoriser**.

Pourquoi deux tables pour une personne ? mcd_users est la table standard de Laravel (identifiants de connexion) ; mcd_utilisateurs est la fiche de l'enseignant (nom, rôle, statut…). Le MCD les relie avec le même identifiant.

4. Les fonctionnalités, une par une

4.1 Inscription (n° 2)

À quoi ça sert. Un visiteur crée son propre compte. Il devient **Candidat**, au statut **Inactif** : un administrateur doit l'activer avant qu'il puisse se connecter.

Fichiers concernés. `config/fortify.php` (fonction `registration()` activée), `FortifyServiceProvider.php` (vue du formulaire, action de création), `CreateNewUser.php`, `RegisterResponse.php`, `resources/views/auth/register.blade.php`, modèles `User`, `Utilisateur`, `Role`, `Genre`.

Que se passe-t-il, étape par étape.

1. Le visiteur ouvre `/register`. La route est fournie par **Fortify** ; notre `FortifyServiceProvider` lui dit d'afficher `auth.register`, avec la liste des genres lue dans mcd_genres.
2. Il envoie le formulaire. Fortify appelle `CreateNewUser::create()`.
3. Les champs sont **validés** : nom et prénom (50 caractères maximum), genre existant, classe facultative, e-mail valide et **unique** dans mcd_users, mot de passe respectant les règles. Si une règle échoue, on revient au formulaire avec un message.
4. On cherche le rôle `CAND` dans mcd_roles.
5. Dans une **transaction** (`DB::transaction`), on crée d'abord le compte (mcd_users) puis la fiche (mcd_utilisateurs) avec le rôle CAND et le statut **I**. La transaction garantit que **les deux sont créés, ou aucun** : on ne peut pas se retrouver avec un compte sans fiche.
6. Par défaut, Fortify **connecte** le nouveau compte. Notre `RegisterResponse` annule cela : il déconnecte, détruit la session et renvoie vers `/login` avec le message « Compte créé, en attente de validation par un administrateur. ».

Tables. mcd_users, mcd_utilisateurs (lecture de mcd_roles et mcd_genres). **Droit.** Aucun (page publique).

Pourquoi le rôle et le statut ne sont jamais dans le formulaire ? Si on les laissait choisir, n'importe qui pourrait s'inscrire comme administrateur. Ils sont donc fixés **par le serveur**.

Comment le tester. S'inscrire avec un nouvel e-mail ; vérifier le message sur la page de connexion ; essayer de se connecter (refusé : « Ce compte est désactivé. ») ; se connecter en administrateur pour activer le compte.

4.2 Connexion et limitation des tentatives (n° 4 et 5)

À quoi ça sert. Vérifier l'identité d'un utilisateur et **refuser** les comptes qui ne sont pas actifs.

Fichiers concernés. `FortifyServiceProvider.php` (fonction `authenticateUsing`), `config/fortify.php`, `lang/fr/auth.php`, `resources/views/auth/login.blade.php`, `resources/views/layouts/auth.blade.php`.

Que se passe-t-il, étape par étape.

1. L'utilisateur ouvre `/login`, remplit e-mail et mot de passe, puis envoie le formulaire.
2. Fortify appelle notre fonction `authenticateUsing`. Elle cherche le compte avec cet e-mail dans mcd_users.
3. **E-mail inconnu ou mot de passe faux** : la fonction renvoie `null`. Fortify affiche le message « Identifiants incorrects. » (`lang/fr/auth.php`). Le message est volontairement vague : il ne dit pas si c'est l'e-mail ou le mot de passe qui est faux.
4. La comparaison se fait avec `Hash::check()` : on compare le mot de passe tapé avec le **hash** en base, jamais avec un mot de passe en clair.
5. **Mot de passe correct** : on lit le statut (`$user->utilisateur?->statut?->code`). Seul le statut **A** passe. Pour **I** : « Ce compte est désactivé. » ; pour tout autre statut (B, ou un statut ajouté plus tard) : « Ce compte est bloqué. ». Si le compte n'a pas de fiche, le code est `null` : la connexion est refusée.
6. Statut **A** : l'utilisateur est connecté et envoyé vers `/menu` (réglage `home` de `config/fortify.php`).

Pourquoi tester « ce qui est autorisé » (`!== 'A'`) plutôt que « ce qui est interdit » ? Si un administrateur ajoute plus tard un statut « Suspendu » dans le Paramétrage, il sera **bloqué par défaut**. Le code reste sûr sans modification.

Limitation des tentatives. Comme `limiters.login` vaut `null` dans `config/fortify.php`, Fortify bloque lui-même un couple **e-mail + adresse IP** après **5 échecs en une minute** et affiche « Trop de tentatives de connexion. Veuillez réessayer dans X secondes. ». Je n'ai rien eu à coder : c'est un réglage.

Tables. mcd_users, mcd_utilisateurs, mcd_statuts. **Droit.** Aucun (page publique).

Comment le tester. Se connecter avec un compte actif (page `/menu`) ; avec un mauvais mot de passe (« Identifiants incorrects. ») ; avec un compte inactif (« Ce compte est désactivé. ») ; se tromper 6 fois de suite (message de limitation).

4.3 Mot de passe oublié et réinitialisation (n° 3 et 7)

À quoi ça sert. Retrouver l'accès à son compte sans l'aide de l'administrateur.

Fichiers concernés. `config/fortify.php` (fonction `resetPasswords()`), `FortifyServiceProvider.php` (vues), `ResetUserPassword.php`, `PasswordValidationRules.php`, `resources/views/auth/forgot-password.blade.php` et `reset-password.blade.php`, table `mcd_password_reset_tokens` (script `ajout_password_reset.sql`), réglages `MAIL_*` du `.env`.

Que se passe-t-il, étape par étape.

1. Sur `/forgot-password`, l'utilisateur saisit son e-mail.
2. Fortify fabrique un **jeton** (une longue suite de caractères secrets), l'enregistre (haché) dans **mcd_password_reset_tokens** et envoie un e-mail contenant un lien du type `/reset-password/<jeton>?email=…`.

3. Le lien est **valable 60 minutes** et **utilisable une seule fois**.
4. L'utilisateur ouvre le lien : Fortify affiche `auth.reset-password`, à qui nous transmettons la requête pour lire le jeton et l'e-mail.
5. Il saisit le nouveau mot de passe (deux fois). `ResetUserPassword::reset()` le valide puis l'enregistre **haché** (`Hash::make`).
6. **Important :** la réinitialisation ne change **que le mot de passe**. Un compte inactif ou banni reste refusé à la connexion (le contrôle du statut est dans `authenticateUsing`).

E-mails. En développement, `MAIL_MAILER=log` : l'e-mail n'est pas envoyé mais écrit dans `storage/logs/laravel.log` (commande utile : `grep "reset-password/" storage/logs/laravel.log | tail -1`). Sur la VM, le serveur SMTP est renseigné dans le `.env` ; avec le port 465 il faut `MAIL_SCHEME=smtps`.

Tables. mcd_users, mcd_password_reset_tokens. **Droit.** Aucun.

4.4 Page Profil et changement du mot de passe (n° 9)

À quoi ça sert. Chaque utilisateur connecté voit les informations de **son** compte (en lecture seule) et peut changer son mot de passe.

Fichiers concernés. `routes/web.php` (route `profil`, protégée par `auth` seulement), `ProfilController.php`, `resources/views/profil/show.blade.php`, `UpdateUserPassword.php`, `FortifyServiceProvider.php`.

Que se passe-t-il, étape par étape.

1. `GET /profil` : `ProfilController::show()` récupère l'utilisateur connecté (`$request->user()`) et charge sa fiche avec son rôle, son statut et son genre (`load('role', 'statut', 'genre')`).
2. La vue affiche ces informations en lecture seule. S'il manque une information, elle affiche « — » (opérateurs `?->` et `??`).
3. Le formulaire de changement de mot de passe est envoyé en `PUT /user/password` (une route de Fortify ; `@method('PUT')` dans la vue, car un formulaire HTML ne sait envoyer que GET ou POST).
4. `UpdateUserPassword::update()` vérifie le **mot de passe actuel** (`current_password:web`) et valide le nouveau (saisi deux fois, règle par défaut de Laravel : 8 caractères minimum), puis l'enregistre haché. Les erreurs sont rangées dans un « sac » nommé `updatePassword` pour ne s'afficher que sur ce formulaire.

Droit. Aucun droit particulier : tout utilisateur connecté (`auth`).

4.5 Menu adapté au rôle (n° 8)

À quoi ça sert. Ne montrer que les liens que le rôle a le droit d'utiliser.

Fichiers concernés. `resources/views/partials/menu.blade.php` (inclus par `layouts/app.blade.php`), `MenuController.php`, `resources/views/menu.blade.php`.

Comment ça marche. Chaque lien est entouré de `@can('action') … @endcan` : il n'est affiché que si le rôle de l'utilisateur possède cette action dans **mcd_autoriser**. Pour un menu déroulant, `@canany([...])` affiche le menu si **au moins une** action est permise (on évite un menu vide). `request()->routeIs('…')` surligne la page courante.

Attention (point clé pour l'oral). Le menu ne fait qu'**afficher ou cacher** des liens : c'est de l'**apparence**. La **vraie protection**, c'est le middleware `can:` de chaque route, côté serveur. Quelqu'un qui tape l'adresse à la main reçoit une erreur 403 même si le lien est caché.

4.6 Liste des utilisateurs (n° 11)

À quoi ça sert. L'administrateur voit les comptes de l'application.

Fichiers concernés. `routes/web.php` (route `utilisateurs.index`, `can:utilisateurs.viewAny`), `UtilisateurController::index()`, `resources/views/admin/utilisateurs/index.blade.php`.

Comment ça marche. La route est protégée par `auth` puis `can:utilisateurs.viewAny`. Le contrôleur renvoie la vue `admin.utilisateurs.index`, qui affiche le composant Livewire `GestionComptes` (partie d'Evan : recherche, filtre, boutons de statut). Ma contribution est la **protection de la page** et le **contrôle des droits** (le composant revérifie aussi les droits avec `Gate::authorize`).

Droit. `utilisateurs.viewAny` (rôle ADMIN).

4.7 Création d'un utilisateur (n° 12)

À quoi ça sert. L'administrateur crée directement un compte avec le **rôle**, le **statut** et le **genre** de son choix (contrairement à l'inscription).

Fichiers concernés. `routes/web.php` (`utilisateurs.create` et `utilisateurs.store`, `can:utilisateurs.create`), `UtilisateurController::create()` et `store()`, `resources/views/admin/utilisateurs/create.blade.php`, modèles `User`, `Utilisateur`, `Role`, `Statut`, `Genre`.

Que se passe-t-il, étape par étape.

1. `create()` envoie à la vue les listes de rôles, statuts et genres (triées par nom) pour remplir les listes déroulantes.
2. `store()` **valide** le formulaire : nom et prénom (50 caractères maximum), e-mail valide et **unique** dans mcd_users, mot de passe de **8 caractères minimum**, et rôle, statut et genre qui **existent vraiment** (`exists:mcd_roles,id`, `exists:mcd_statuts,code`, `exists:mcd_genres,code`).
3. Les messages d'erreur sont écrits en français (« Cette adresse email est déjà utilisée par un autre compte. »).
4. On crée le compte (`User::create`), puis la fiche avec `$user->utilisateur()-`

`>create([...])` : Laravel donne automatiquement à la fiche **le même identifiant** que le compte.

5. Retour à la liste avec le message « Utilisateur créé avec succès. ».

Droit. `utilisateurs.create` (rôle ADMIN).

4.8 Paramétrage : rôles, statuts, genres (n° 14)

À quoi ça sert. Ajouter une valeur aux tables de référence sans toucher à la base à la main.

Fichiers concernés. `routes/web.php` (`parametrage.index`, `parametrage.roles.store`, `parametrage.statuts.store`, `parametrage.genres.store`), `ParametrageController.php`, `resources/views/admin/parametrage.blade.php`, modèles `Role`, `Statut`, `Genre`.

Que se passe-t-il, étape par étape.

1. `index()` lit les trois tables (rôles, statuts, genres) et les affiche dans trois tableaux, chacun avec son formulaire d'ajout.
2. `storeRole`, `storeStatut`, `storeGenre` **valident** : code et nom **uniques** ; pour un statut ou un genre, le code fait **exactement 1 lettre** ; pour un rôle, le code fait 10 caractères au plus.
3. Le code est enregistré en **majuscules** (`strtoupper`).
4. Retour à la page avec un message, par exemple « Rôle « Securité » ajouté. ».

Pas de suppression, volontairement : ces valeurs sont utilisées par des **clés étrangères** (une fiche pointe vers un rôle, un statut et un genre). Supprimer une valeur utilisée casserait des comptes.

Droits. `parametrage.viewAny` (afficher la page) et `parametrage.create` (ajouter), rôle ADMIN.

Point d'attention. Un rôle ajouté ici n'a **aucun droit** tant qu'on n'a pas ajouté de lignes dans **mcd_autoriser** (voir 5.4).

5. Les autorisations (n° 23, 24 et 25) : le cœur de ma partie

5.1 Les rôles (n° 24)

Cinq rôles : **ADMIN** (administrateur), **GEST** (gestionnaire des épreuves), **CAND** (candidat), **JURY_ENS** (jury enseignant) et **JURY_PRO** (jury professionnel). À l'origine il n'y avait qu'un rôle JURY ; il a été scindé en deux sur décision de l'enseignant (script de migration `migration_roles_jury.sql` pour une base déjà remplie).

Le **rôle** dit **ce que l'on a le droit de faire**. Le **statut** (5.2) dit **si l'on a le droit de se connecter**.

5.2 Les statuts (n° 25)

Code	Nom	Effet
A	Actif	La connexion est permise.
I	Inactif	Connexion refusée : « Ce compte est désactivé. ». Statut de tout compte créé par l'inscription.
B	Banni	Connexion refusée : « Ce compte est bloqué. ».

5.3 Le principe : des droits dans la base, pas dans le code

Au lieu d'écrire dans le code « si le rôle est GEST, alors… », les droits sont des **lignes de la base** :

- **mcd_actions** liste les 13 actions possibles (`epreuves.create`, `utilisateurs.update`…) ;
- **mcd_autoriser** dit quel rôle a quelle action.

Pour changer **qui peut faire quoi**, on modifie des lignes de la base, **sans toucher au programme**.

Le trajet complet d'une vérification (exemple : un gestionnaire ouvre `/gestion/epreuves`) :

1. **Au démarrage de Laravel** (`AppServiceProvider::boot()` → `definirLesGates()`), on lit les noms de la table mcd_actions et on crée une **Gate** par action : `Gate::define('epreuves.viewAny', …)`. Une Gate est une règle nommée qui répond vrai ou faux pour l'utilisateur connecté.
2. La requête arrive. Dans `routes/web.php`, la route est protégée par deux **middleware** dans cet ordre : `auth` (est-il connecté ? sinon redirection vers `/login`), puis `can:epreuves.viewAny` (a-t-il le droit ?).
3. `can:` appelle la Gate, qui appelle `$user->hasPermission('epreuves.viewAny')`.
4. `User::hasPermission()` passe par la fiche (`Utilisateur::hasPermission()`), qui pose **une seule question SQL** : « le rôle de cet utilisateur a-t-il une ligne dans mcd_autoriser pour cette action ? ».
5. Vrai : la page s'affiche. Faux : erreur **403** (« This action is unauthorized. »).

Le code central, avec son explication :

```
// AppServiceProvider.php : au démarrage, une Gate par action de mcd_actions
$nomsDesPermissions = Permission::pluck('nom');   // ['epreuves.create', ...]
foreach ($nomsDesPermissions as $permission) {
    Gate::define($permission, function (User $user) use ($permission) {
        return $user->hasPermission($permission);
    });
}
// Utilisateur.php : une seule requête SQL pour décider
public function hasPermission(string $permission): bool
{
    return $this->role()                        // le rôle de l'utilisateur
        ->whereHas('permissions', function ($requete) use ($permission) {
```

```
            $requete->where('nom', $permission);   // a-t-il cette action ?
        })
        ->exists();                             // vrai ou faux
}
// routes/web.php : auth d'abord, can: ensuite
Route::middleware('auth')->group(function () {
    Route::get('/admin/utilisateurs', [UtilisateurController::class,
'index'])
        ->name('utilisateurs.index')
        ->middleware('can:utilisateurs.viewAny');
});
```

Deux moments à ne pas confondre. Au **démarrage**, Laravel ne lit que **mcd_actions** (pour créer les Gates). C'est **à chaque vérification** qu'il lit **mcd_autoriser** (via `hasPermission`). Conséquence : un changement dans mcd_autoriser est pris en compte **immédiatement**.

Protection en plusieurs couches :

Couche	Outil	Rôle
Route	`auth` puis `can:`	Protection réelle, côté serveur.
Vue	`@can`, `@canany`	Cache les liens non permis (apparence seulement).
Livewire	`Gate::authorize()`	Revérifie dans `mount()` et dans **chaque action**, car chaque clic est une nouvelle requête qui ne repasse pas par le middleware de la route.

Cas particulier à connaître. Si la base est inaccessible ou si la table mcd_actions n'existe pas encore au démarrage, `definirLesGates()` est entouré d'un `try / catch` : l'erreur est notée dans `storage/logs/laravel.log` et aucune Gate n'est créée. Les pages protégées répondent alors **403 pour tout le monde, y compris l'administrateur**. Sans ce `try / catch`, des commandes comme `composer install` planteraient.

5.4 La matrice des droits (13 actions, 15 droits)

Action	ADMIN	GEST	CAND	JURY_ENS	JURY_PRO
utilisateurs.viewAny	✔				
utilisateurs.create	✔				
utilisateurs.update	✔				
parametrage.viewAny	✔				
parametrage.create	✔				
epreuves.viewAny		✔			
epreuves.create		✔			

Action	ADMIN	GEST	CAND	JURY_ENS	JURY_PRO
epreuves.update		✔			
lieux.create		✔			
convocations.viewAny		✔			
convocations.view			✔		
planning.viewAny				✔	✔
evaluations.viewAny				✔	✔

5.5 Comment modifier un droit

Retirer ou donner un droit à un rôle (exemple : donner `lieux.create` au rôle JURY_ENS) : ajouter une ligne dans mcd_autoriser. Prends modèle sur les insertions de `init.sql` (les colonnes de dates peuvent être remplies par des triggers).

```
INSERT INTO mcd_autoriser (id_role, id_action)
SELECT r.id, a.id
FROM mcd_roles r, mcd_actions a
WHERE r.code = 'JURY_ENS' AND a.nom = 'lieux.create';
```

Créer une nouvelle action : l'ajouter dans mcd_actions (nom au format `ressource.action`), puis l'utiliser dans une route avec `->middleware('can:nom.de.l.action')` et dans les vues avec `@can`. La Gate correspondante est créée automatiquement au prochain démarrage (donc à la prochaine requête). **Une action absente de mcd_actions n'a pas de Gate : l'accès est refusé à tout le monde.**

Donner des droits à un nouveau rôle : après l'avoir créé dans la page Paramétrage, ajouter ses lignes dans mcd_autoriser (sinon il n'a accès à rien d'autre qu'à `/menu` et `/profil`).

6. Sécurité : ce que ma partie garantit

Risque	Protection mise en place	Où
Mot de passe volé dans la base	Mots de passe **hachés** (bcrypt) ; comparaison avec `Hash::check` ; cast `hashed` dans `User`.	`User.php`, `FortifyServiceProvider.php`
Devenir administrateur en s'inscrivant	Rôle **CAND** et statut **I** fixés par le serveur.	`CreateNewUser.php`
Compte non validé qui se connecte	Contrôle du statut (seul **A** passe).	`FortifyServiceProvider.php`
Deviner un mot de passe par force brute	**5 essais par minute** par couple e-mail + IP.	`config/fortify.php`
Savoir si un e-mail existe	Message volontairement vague «	`lang/fr/auth.php`

Risque	Protection mise en place	Où
	Identifiants incorrects. ».	
Accéder à une page par son adresse	`auth` puis `can:` sur **chaque** route, côté serveur.	`routes/web.php`
Contourner les droits via Livewire	`Gate::authorize()` à chaque action.	`GestionComptes.php`
Compte sans fiche (incohérence)	Création du compte et de la fiche en **transaction**.	`CreateNewUser.php`
Données invalides	Validation côté serveur (`unique`, `exists`, `size`, `min`).	contrôleurs et actions Fortify
Fuite des identifiants de la base	`.env` exclu de Git, permissions `640` sur la VM.	`deploy.sh`
Interception des échanges	**HTTPS** avec certificat d'une autorité locale.	Apache (section 8.4)

Limites connues (à dire honnêtement) : un compte **banni pendant qu'il est déjà connecté** reste actif jusqu'à sa déconnexion (le statut n'est vérifié qu'à la connexion) ; l'inscription est ouverte à tous (pas de captcha) ; le certificat HTTPS vient d'une autorité **locale** : les autres navigateurs affichent « Non sécurisé » tant que son certificat n'y est pas importé.

7. La documentation (n° 27)

Document	Contenu	Où
`README.md`	Installation, rôles, authentification, organisation Git, limites connues, comptes de test.	racine du dépôt GitHub
`docs/architecture.md`	Schéma fonctionnel, schéma technique, matrice des droits.	dossier `docs/` du dépôt
Ce document	Fonctionnement et fichiers de ma partie.	dossier `docs/` du dépôt et dossier E6
Documentation de déploiement (version 2.0)	Installation pas à pas sur la machine virtuelle.	fournie avec le dossier

8. Le déploiement (n° 29)

8.1 Vue d'ensemble

L'application tourne sur une **machine virtuelle Proxmox** (Debian 13) avec **Apache**, **PHP-FPM** et **MariaDB**. Elle est accessible depuis le réseau du lycée à l'adresse `https://172.16.100.94/`.

Image à retenir : un restaurant. **Apache** est le serveur en salle (il reçoit les clients), **PHP-FPM** est la cuisine (il exécute le code Laravel), **MariaDB** est la réserve (les données), et le dossier `current` est le **menu du jour**, qu'on remplace en changeant d'étiquette.

8.2 Le principe « releases + lien symbolique »

Chaque déploiement crée un **nouveau dossier daté** dans `/var/www/releases/` (par exemple `a1b2c3d4-20261104-101500`). Un **lien symbolique** (un raccourci) nommé `current` pointe vers la version active ; Apache sert toujours `current/public`. Pour changer de version, on **déplace le raccourci** : c'est instantané, et on peut **revenir en arrière** en le remettant sur l'ancienne version. Les **5 dernières versions** sont conservées.

8.3 Ce que fait `deploy.sh`

Le script se lance avec `sudo bash deploy.sh`. Il refuse de démarrer sans `sudo` ou sans le fichier `.env` de l'utilisateur. Ses étapes :

1. Lire sur GitHub le dernier commit de la branche `main` et fabriquer un nom de version (début du commit + date et heure).
2. Cloner le dépôt dans `/var/www/releases/<version>`.
3. Copier le `.env` de l'utilisateur dans la nouvelle version.
4. `composer install --no-dev --optimize-autoloader` : installer les dépendances (sans les outils de développement).
5. `php artisan optimize:clear`, puis `config:cache`, `route:cache`, `view:cache` : accélérer l'application.
6. **Jamais de** `php artisan migrate` : la structure de la base vient des scripts SQL.
7. Régler les **permissions** : dossiers en 755, fichiers en 644, `storage` et `bootstrap/cache` en 775 (Laravel doit y écrire), `.env` en 640 (illisible par les autres).
8. Faire pointer `current` vers la nouvelle version, tester la configuration Apache (`apache2ctl configtest`) et recharger Apache.
9. Supprimer les versions au-delà des 5 plus récentes.

8.4 Les autres éléments

- `setup_db_vm.sh` (première installation seulement) : crée la base `ccf_convocations` et l'utilisateur `ccf_user`, importe `creation.sql`, `init.sql`, `test.sql` et `ajout_password_reset.sql`, et génère le `.env`. Le mot de passe de la base est **saisi au clavier** (12 caractères minimum), jamais écrit dans un fichier du dépôt.
- `ccf-convocations.conf` : le site Apache. `DocumentRoot` vaut `/var/www/releases/current/public`, `AllowOverride All` autorise le fichier `.htaccess` de Laravel, et PHP est transmis à **PHP-FPM** par une socket.
- **HTTPS** : une **autorité de certification locale** a signé le certificat du serveur (certificat pour le serveur seulement, avec l'adresse IP parmi ses noms valides). Le site écoute sur le port 443 et le port 80 **redirige** vers HTTPS en conservant l'adresse demandée. Pour qu'un navigateur n'affiche pas « Non sécurisé », il faut **importer le certificat de**

l'autorité (fichier `ca-ccf.crt`) dans ce navigateur.

- **E-mails** : serveur SMTP renseigné dans le `.env` (`MAIL_*`), avec `MAIL_SCHEME=smtps` pour le port 465.

8.5 Mettre à jour et revenir en arrière

- **Nouvelle version** : `git push` sur la branche, Pull Request fusionnée dans `main`, puis sur la VM `sudo bash deploy.sh`.
- **Retour arrière** : remettre `current` sur la version précédente puis recharger Apache :

```
ls -1dt /var/www/releases/????????-????????-??????     # lister les versions
sudo ln -sfn /var/www/releases/<ancienne_version> /var/www/releases/current
sudo systemctl reload apache2
```

- **Sauvegarde de la base** : voir la documentation de déploiement (section « Mises à jour et retour arrière »).

9. Tester ma partie : scénarios à dérouler

N°	Scénario	Résultat attendu
1	Ouvrir `/` sans être connecté	Page d'accueil ; si déjà connecté, redirection vers `/menu`.
2	Ouvrir `/menu` sans être connecté	Redirection vers `/login` (middleware `auth`).
3	Se connecter avec un mauvais mot de passe	« Identifiants incorrects. »
4	Se tromper 6 fois de suite	Message de limitation, avec le nombre de secondes d'attente.
5	S'inscrire, puis tenter de se connecter	Message « Compte créé, en attente… », puis « Ce compte est désactivé. »
6	Administrateur : activer le compte, puis se reconnecter avec	Connexion acceptée ; le nouveau compte est un Candidat.
7	Mot de passe oublié avec un e-mail existant	Message de confirmation ; lien dans l'e-mail (ou dans `laravel.log` en développement).
8	Ouvrir deux fois le même lien de réinitialisation	La seconde fois, le lien est refusé (usage unique).
9	Profil : changer le mot de passe avec un mauvais « mot de passe actuel »	« Le mot de passe actuel est incorrect. »
10	Un jury enseignant ouvre `/admin/utilisateurs`	Erreur **403**.
11	Un candidat regarde le menu	Il ne voit que « Accueil », « Ma convocation » et son profil.
12	Administrateur : créer un utilisateur avec un	« Cette adresse email est déjà utilisée par un

N°	Scénario	Résultat attendu
	e-mail déjà utilisé	autre compte. »
13	Administrateur : ajouter un statut avec un code de 2 lettres	« Le code doit faire exactement 1 lettre. »
14	Administrateur : ajouter un rôle déjà existant	« Ce code de rôle existe déjà. »

10. Dépannage ciblé

Problème	Cause probable	Que faire
Toutes les pages protégées répondent 403, **même pour l'administrateur**	Table mcd_actions ou mcd_autoriser absente ou vide (donc aucune Gate).	Regarder `storage/logs/laravel.log` (message « Autorisations non chargées… »), puis importer `ajout_autorisations.sql` ou vérifier `init.sql`.
Un rôle n'a pas accès à une page prévue	Ligne manquante dans mcd_autoriser.	Vérifier la ligne (section 5.5).
« Ce compte est désactivé. » alors qu'on attend un accès	Statut `I` du compte.	L'administrateur active le compte (page des utilisateurs).
Le lien de réinitialisation n'arrive pas	`MAIL_MAILER=log` (développement) ou SMTP mal réglé.	Chercher le lien dans `laravel.log`, ou vérifier `MAIL_*` dans le `.env`.
Erreur 500 après un déploiement	`.env` manquant, ou permissions de `storage` et `bootstrap/cache`.	Relire `storage/logs/laravel.log` ; relancer `deploy.sh`.
Les modifications du code ne s'affichent pas	Caches de Laravel.	`php artisan optimize:clear`.
« Non sécurisé » dans le navigateur	Certificat de l'autorité locale non importé.	Importer `ca-ccf.crt` dans ce navigateur.

Glossaire

Mot	Définition simple
Route	Une adresse du site (`/login`) associée à un contrôleur.
Contrôleur	Classe PHP qui reçoit la demande et prépare la réponse.
Modèle (Eloquent)	Classe PHP qui représente une table de la base.
Vue (Blade)	Fichier qui fabrique la page HTML.
Middleware	Contrôle exécuté avant le contrôleur (`auth`, `can:`).
Gate	Règle nommée qui répond vrai ou faux pour l'utilisateur connecté.
Fortify	Module de Laravel qui fournit connexion, inscription et mot de passe oublié, sans interface.
Livewire	Module qui met à jour une partie de la page sans la recharger.
Hachage (bcrypt)	Transformation à sens unique d'un mot de passe : on ne peut pas retrouver le mot de passe à partir du hash.
Transaction	Groupe d'écritures en base : tout réussit ou tout est annulé.
Clé étrangère	Colonne qui pointe vers une ligne d'une autre table.
Association non fonctionnelle	Association plusieurs-à-plusieurs du MCD ; elle devient une table de liaison.
Release	Version déployée, dans son propre dossier daté.
Lien symbolique	Raccourci vers un dossier (`current` vers la version active).
.env	Fichier de réglages propre à chaque machine, avec des secrets ; jamais dans Git.